

CPU vs. GPU: A Comparative Analysis of Architecture, Parallelism, and Workload Fit

Aksel Aghajanyan

Technical Report — Knowledge Base Archive

Abstract

Central Processing Units (CPUs) and Graphics Processing Units (GPUs) represent two distinct answers to the question of how silicon should be organized to execute computations efficiently. CPUs optimize for low-latency, general-purpose execution on irregular, branch-heavy workloads through a small number of powerful cores, deep cache hierarchies, and sophisticated control logic. GPUs optimize for high-throughput execution on data-parallel workloads through thousands of lightweight cores organized in Single-Instruction, Multiple-Thread (SIMT) groups, massive memory bandwidth, and latency-hiding schedulers. This report provides a mathematically grounded comparison of CPU and GPU architectures, parallelism models, memory subsystems, and the workload characteristics that determine which processor class—or which hybrid partitioning—delivers superior performance.

I. INTRODUCTION

The phrase “run it on the GPU” has become commonplace in machine learning, scientific computing, and real-time graphics. Yet the choice between CPU and GPU execution is not a matter of universal acceleration: it is an engineering decision grounded in **workload structure**, **memory access patterns**, and **parallelism granularity**.

A CPU is designed as a **latency-optimized** processor. Its cores execute complex instruction streams with unpredictable branches, pointer chasing, and frequent synchronization. A GPU is designed as a **throughput-optimized** processor. Its cores execute the same instruction across large arrays of data elements, tolerating long memory latencies by switching among thousands of resident threads.

Confusion arises when practitioners treat GPUs as “faster CPUs.” They are not interchangeable. A sequential algorithm with poor data parallelism may execute *slower* on a GPU than on a modern CPU due to kernel launch overhead, PCIe transfer costs, and occupancy limitations. Conversely, dense matrix multiplication, convolution, ray tracing, and particle simulations routinely achieve order-of-magnitude speedups on GPUs because their arithmetic intensity and regular memory access patterns align with the GPU execution model.

This report compares CPU and GPU architectures across four axes: core organization and control flow, parallelism and scaling laws, memory hierarchy and bandwidth, and workload fit criteria including the roofline performance model.

II. ARCHITECTURAL FOUNDATIONS

A. CPU: Latency-Optimized General-Purpose Cores

A modern CPU die allocates substantial area to features that minimize per-thread latency:

- **Few powerful cores** (typically 8–64 on consumer and server parts), each capable of superscalar, out-of-order execution at 3–5 GHz.
- **Deep cache hierarchy** — private L1/L2 caches per core, shared L3, and hardware prefetchers that hide DRAM latency for sequential and strided access.
- **Branch prediction and speculative execution** — critical for code with conditional logic, virtual function calls, and irregular control flow.
- **Low-latency interconnect** between cores, memory controllers, and I/O subsystems.

The CPU execution model is best described as **Multiple-Instruction, Multiple-Data (MIMD)**: independent threads may follow entirely different instruction paths. This generality makes the CPU the default host for operating systems, databases, web servers, compilers, and any workload dominated by serial or fine-grained irregular parallelism.

B. GPU: Throughput-Optimized Streaming Processors

A modern GPU die inverts the CPU design priorities:

- **Thousands of simple cores** grouped into Streaming Multiprocessors (SMs) or Compute Units (CUs), clocked at 1–2 GHz.
- **SIMT execution** — threads are scheduled in **warps** (NVIDIA, 32 threads) or **wavefronts** (AMD, 64 threads) that share a program counter at instruction issue; divergence is handled via masking.
- **Massive register files and shared memory** per SM, enabling many warps to remain resident simultaneously.
- **High-bandwidth memory (HBM or GDDR)** with aggregate throughput exceeding 1 TB/s on datacenter accelerators, at the cost of higher access latency than CPU DRAM paths.

GPUs were originally architected for rasterization and shading pipelines—embarrassingly parallel workloads over millions of pixels. General-purpose GPU (GPGPU) computing, formalized by CUDA (2007) and OpenCL, repurposed this hardware for linear algebra, FFTs, Monte Carlo simulation, and neural network training.

C. Design Philosophy in One Equation

Let P denote peak compute throughput (FLOP/s), f clock frequency, C core count, and I instructions per cycle per core. Roughly:

$$P_{\text{CPU}} \approx f_{\text{CPU}} \cdot C_{\text{CPU}} \cdot I_{\text{CPU}}, \quad I_{\text{CPU}} \gg I_{\text{GPU}} \quad (1)$$

$$P_{\text{GPU}} \approx f_{\text{GPU}} \cdot C_{\text{GPU}} \cdot I_{\text{GPU}}, \quad C_{\text{GPU}} \gg C_{\text{CPU}} \quad (2)$$

CPUs win on I (IPC, instruction-level parallelism per core); GPUs win on C (core count). Neither dominates universally—the winning architecture depends on whether the workload can supply enough independent work to saturate C_{GPU} .

III. PARALLELISM MODELS AND SCALING LAWS

A. Data Parallelism and SIMT

A data-parallel kernel maps N independent work items to N threads. On a GPU, thread i computes:

$$y[i] = f(x[i]), \quad i \in \{0, 1, \dots, N-1\} \quad (3)$$

When f is identical across all i and branch divergence is minimal, SIMT hardware issues one instruction for an entire warp, amortizing decode and dispatch overhead across 32 threads.

Warp divergence occurs when threads within a warp take different branches:

$$y[i] = \begin{cases} f_1(x[i]) & \text{if } c[i] \\ f_2(x[i]) & \text{otherwise} \end{cases} \quad (4)$$

The hardware serializes the taken paths, reducing effective throughput. CPUs handle divergent branches natively via branch prediction; GPUs penalize them heavily.

B. Amdahl's Law

For a program with serial fraction s and parallel fraction $p = 1 - s$, speedup on N processors is bounded by:

$$S(N) = \frac{1}{s + \frac{p}{N}} \xrightarrow{N \rightarrow \infty} \frac{1}{s} \quad (5)$$

Even a small serial section ($s = 0.01$) caps speedup at $100\times$ regardless of GPU size. CPU code paths for setup, I/O, reduction, and host-side orchestration often constitute this serial fraction. Hybrid CPU–GPU pipelines must minimize s through batching, asynchronous transfers, and overlapping compute with data movement.

C. Gustafson’s Law

For scaled problems where parallel work grows with processor count:

$$S(N) = N - s \cdot (N - 1) \quad (6)$$

Large-batch neural network training exemplifies Gustafson scaling: increasing GPU count permits larger minibatches and bigger models, partially offsetting serial overhead. The applicable law depends on whether problem size is **fixed** (Amdahl) or **scaled** (Gustafson).

D. Occupancy and Latency Hiding

GPU throughput requires sufficient **occupancy** — the ratio of active warps to maximum supported warps per SM. If a kernel stalls on memory (high latency L_{mem}), the scheduler hides latency only if enough other warps are ready:

$$\text{Effective throughput} \propto \min\left(P_{\text{peak}}, \frac{B_{\text{mem}}}{\text{AI}}\right) \quad (7)$$

where B_{mem} is memory bandwidth and AI is arithmetic intensity (FLOP per byte transferred). Low-occupancy kernels with irregular memory access fail to hide latency and underutilize the device.

IV. MEMORY HIERARCHY AND DATA MOVEMENT

A. CPU Memory Subsystem

CPUs prioritize **low latency**. Typical hierarchy:

- 1) L1 cache (~ 1 ns, 32–64 KB per core)
- 2) L2 cache (~ 3 –10 ns, 256 KB–1 MB per core)
- 3) L3 cache (~ 10 –40 ns, 8–64 MB shared)
- 4) DRAM (~ 50 –100 ns, 50–100 GB/s per socket)

Hardware prefetchers and cache coherence protocols (MESI and variants) sustain performance on workloads with temporal and spatial locality. Random pointer chasing — common in trees, hash tables, and object graphs — defeats prefetching and favors CPU architectures with large caches.

B. GPU Memory Subsystem

GPUs prioritize **bandwidth**. Thread-visible memory spaces include:

- **Registers** — per-thread, fastest, private.
- **Shared memory / L1** — per-SM, programmer-managed scratchpad for cooperative loads.
- **L2 cache** — device-wide, helps reuse across SMs.
- **Global memory (HBM/GDDR)** — high bandwidth (hundreds of GB/s to >1 TB/s), high latency (~ 400 –800 cycles).
- **Host memory** — accessed via PCIe/NVLink; transfer cost often dominates small kernels.

The host-to-device transfer time for a buffer of size D bytes is approximately:

$$T_{\text{transfer}} = \frac{D}{B_{\text{PCIe}}} + \tau_{\text{launch}} \quad (8)$$

where B_{PCIe} is link bandwidth (e.g., ~ 32 GB/s for PCIe 4.0 $\times 16$) and τ_{launch} is kernel launch latency (~ 5 –20 μs). Kernels must perform sufficient compute per transferred byte to amortize τ_{launch} and PCIe overhead.

C. Unified Memory and Zero-Copy

Modern platforms offer Unified Memory (CUDA) and shared virtual addressing, allowing the OS and runtime to migrate pages between host and device on demand. These abstractions simplify programming but do not eliminate the physical cost of data movement; explicit buffer management and pinned (page-locked) host memory remain essential for peak performance.

V. WORKLOAD FIT: WHEN TO USE CPU VS. GPU

A. The Roofline Model

The roofline model plots attainable performance as a function of arithmetic intensity:

$$P = \min(P_{\text{peak}}, B_{\text{mem}} \cdot \text{AI}) \quad (9)$$

where $\text{AI} = \frac{\text{FLOP}}{\text{bytes moved}}$.

- **Memory-bound** kernels (AI below the ridge point) are limited by bandwidth — GPUs excel when $B_{\text{mem,GPU}} \gg B_{\text{mem,CPU}}$.
- **Compute-bound** kernels (AI above the ridge point) are limited by peak FLOP/s — the higher $P_{\text{peak,GPU}}$ of tensor cores and wide SIMD units dominates.

The ridge point for a processor is $\text{AI}_{\text{ridge}} = P_{\text{peak}}/B_{\text{mem}}$. GPUs typically have a lower ridge point than CPUs (favoring lower-AI kernels) but a higher ceiling for dense linear algebra.

B. CPU-Favorable Workloads

- Low parallelism or small problem sizes (N too small to fill GPU).
- Irregular control flow, recursion, and frequent synchronization.
- Latency-sensitive tasks (interactive UI, request handling, transaction processing).
- Complex data structures with poor spatial locality (graphs, databases, interpreters).
- Pipelines where PCIe transfer dominates total runtime.

C. GPU-Favorable Workloads

- Large-scale dense linear algebra (GEMM, convolutions, FFTs).
- Embarrassingly parallel simulation (particles, Monte Carlo, image processing).
- Neural network forward and backward passes (especially with mixed-precision tensor cores).
- Rendering, video encoding/decoding (when hardware codecs are unavailable or custom kernels are needed).
- Sustained batch processing where amortized transfer cost is negligible.

D. Hybrid CPU–GPU Execution

Production systems routinely partition work across both:

$$T_{\text{total}} = T_{\text{CPU,serial}} + T_{\text{transfer}} + T_{\text{GPU,parallel}} + T_{\text{sync}} \quad (10)$$

Effective hybrid design overlaps T_{transfer} with $T_{\text{GPU,parallel}}$ via CUDA streams or asynchronous queues, keeps hot data resident on device across iterations, and reserves the CPU for scheduling, preprocessing, and reductions ill-suited to SIMT hardware.

VI. CASE STUDIES IN COMPARATIVE PERFORMANCE

A. Dense Matrix Multiplication

For $C = A \times B$ with $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, naive FLOP count is $2MNK$. Optimized GPU GEMM (cuBLAS, rocBLAS) achieves near-peak throughput by tiling into shared memory and exploiting tensor cores. CPUs (Intel MKL, OpenBLAS with AVX-512) remain competitive for small and medium matrices where GPU launch and transfer overhead dominate.

B. Neural Network Training

A training step involves forward pass, loss computation, and backward pass (gradient computation). The dominant operations — matrix multiplications and convolutions — are GPU-native. However, dataloader preprocessing, optimizer state management, and checkpointing often run on CPU. Large-language-model training distributes across **multi-GPU** clusters with CPU orchestration, illustrating that the CPU remains the control plane even in GPU-centric workloads.

C. Video Encoding (HEVC and Successors)

As discussed in related work on HEVC, encoder motion search is combinatorially expensive. Software encoders parallelize across frames and CTUs on CPUs; hardware encoders embed ASIC or fixed-function blocks on GPUs and dedicated media engines. Decode is lighter and increasingly offloaded to fixed-function units on both CPU-integrated graphics and discrete GPUs. The CPU–GPU boundary here is defined by **ASIC specialization**, not raw SIMT generality.

VII. CONCLUSION

CPUs and GPUs are complementary, not competing, architectures. The CPU provides low-latency, general-purpose execution with sophisticated control flow and cache hierarchies optimized for irregular, serial, and latency-sensitive tasks. The GPU provides high-throughput, data-parallel execution with massive bandwidth and SIMT scheduling optimized for regular, arithmetic-intensive kernels at scale.

The decision criterion is not “which is faster” but “which matches the workload’s parallelism, arithmetic intensity, and memory access pattern.” Amdahl’s and Gustafson’s laws bound scaling; the roofline model identifies whether a kernel is compute- or memory-bound; transfer and launch overhead determine whether offloading to the GPU is profitable at all.

Modern computing increasingly adopts **heterogeneous systems**: CPUs manage control, I/O, and irregular logic; GPUs (and TPUs, FPGAs, and media ASICs) accelerate structured parallel phases. Understanding the architectural tradeoffs—latency versus throughput, MIMD versus SIMT, cache versus bandwidth—is essential for principled system design in data science, graphics, scientific computing, and real-time media processing.

REFERENCES

- [1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
- [2] NVIDIA Corporation, “NVIDIA Ampere Architecture Whitepaper,” 2020.
- [3] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [4] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,” in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.